

```

# =====
# Overfitting Example with Dropout
# Google Colab - Error Free Code
# =====

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

print("TensorFlow Version:", tf.__version__)

# -----
# Problem: Model without Dropout
# -----

print("\n===== MODEL WITHOUT DROPOUT =====")

model_without_dropout = Sequential([
    tf.keras.Input(shape=(10,)),
    Dense(128, activation='relu'),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])

model_without_dropout.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_without_dropout.summary()

print("\nProblem:")
print("Large model may memorize the training data and cause overfitting.")

# -----
# Solution: Model with Dropout
# -----

print("\n===== MODEL WITH DROPOUT =====")

model_with_dropout = Sequential([
    tf.keras.Input(shape=(10,)),
    Dense(128, activation='relu'),
    Dropout(0.5),      # 50% neurons are temporarily switched off

    Dense(64, activation='relu'),
    Dropout(0.3),      # 30% neurons are temporarily switched off

    Dense(1, activation='sigmoid')
])

model_with_dropout.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model_with_dropout.summary()

print("\nSolution:")
print("Dropout reduces overfitting by randomly disabling neurons during training.")

print("\nComparison:")
print("Without Dropout -> All neurons work -> Model may memorize data.")
print("With Dropout -> Some neurons are switched off -> Model learns general patterns.")

```

TensorFlow Version: 2.20.0

===== MODEL WITHOUT DROPOUT =====

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,408
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 1)	129

Total params: 18,049 (70.50 KB)

Trainable params: 18,049 (70.50 KB)

Non-trainable params: 0 (0.00 B)

Problem:

Large model may memorize the training data and cause overfitting.

===== MODEL WITH DROPOUT =====

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	1,408
dropout (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 9,729 (38.00 KB)

Trainable params: 9,729 (38.00 KB)

Non-trainable params: 0 (0.00 B)

Solution:

Dropout reduces overfitting by randomly disabling neurons during training.

Comparison:

```
# =====
# Underfitting Example
# Google Colab - Error Free Code
# =====

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

print("TensorFlow Version:", tf.__version__)

# -----
# Problem: Small Model (Underfitting)
# -----

print("\n===== MODEL WITH UNDERFITTING =====")

model_small = Sequential([
    tf.keras.Input(shape=(10,)),
    Dense(1, activation="sigmoid")
])

model_small.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_small.summary()

print("\nProblem:")
print("The model is too small and may underfit the data.")

# -----
# Solution: Larger Model
# -----
```

```

print("\n==== LARGER MODEL =====")

model_large = Sequential([
    tf.keras.Input(shape=(10,)),
    Dense(64, activation="relu"),
    Dense(32, activation="relu"),
    Dense(1, activation="sigmoid")
])

model_large.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_large.summary()

print("\nSolution:")
print("Use more neurons, more hidden layers, and train for more epochs.")

print("\nExample Training:")
print("model.fit(X_train, y_train, epochs=50)")

print("\nComparison:")
print("Small Model -> Underfitting (cannot learn complex patterns)")
print("Larger Model -> Learns patterns more effectively")

```

TensorFlow Version: 2.20.0

==== MODEL WITH UNDERFITTING =====

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 1)	11

Total params: 11 (44.00 B)
 Trainable params: 11 (44.00 B)
 Non-trainable params: 0 (0.00 B)

Problem:

The model is too small and may underfit the data.

==== LARGER MODEL =====

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_7 (Dense)	(None, 64)	704
dense_8 (Dense)	(None, 32)	2,080
dense_9 (Dense)	(None, 1)	33

Total params: 2,817 (11.00 KB)
 Trainable params: 2,817 (11.00 KB)
 Non-trainable params: 0 (0.00 B)

Solution:

Use more neurons, more hidden layers, and train for more epochs.

Example Training:

model.fit(X_train, y_train, epochs=50)

Comparison:

Small Model -> Underfitting (cannot learn complex patterns)